1/06/2019

Basics of python

Q.) what is python?

Ans:-python is general purpose, high level programming language.

Q.) what is the purpose of any programming language?

Ans:- to develop software

Q.) where is python mostly used?

Ans:- 1. Website development

2. data science application

Q.) explain about python and Cpython?

Ans:-python is a programming language.

- Cpython is platform for python program.
- Cpython written in C hence the name Cpython.
- Platform is nothing but runtime infrastructure (execution environment) that support and provide services at the execution time.

Q.) what are the system requirement to develop & execution of python program

Ans:- install Cpython software

Q.) write a python program to print hello world?

Ans:-print(“hello world”)

Q.) who executes python program?

Ans:-python interpreter execute python program.

Q.) how to execute python program?

Ans:-using py command.

Step1:- open command prompt.

Step2:- make that directory as the current working directory in which source file is stored.

Step3:- at cmd give py command after file name and extension.

Q.) what is happening in the background when py command is execute?

Ans:- py command is given to OS

OS call the python interpreter
Python interpreter execute the python program

Q.) how to print the data on the console?

Ans:- using print() library function.

Q.) what are the features of python

Ans:-

- **Simple**
- **Open source and free ware**
- **Interpreted**
- **Extensible**
- **Embedded**
- **Extensive library**
- **Multi-paradigm support**
- **Dynamically typed**

- Portable
- Platform independent

2/06/2019

Q1) who is the creator of python?

--> guido van rossam

Q2) what is the main purpose of computer program?

--> data processing

Q3) can we process data without storing data in computer memory?

--> no.

--> data should be stored in computer memory(RAM) in order for processing.

Q4) how to store data in computer memory in computer program?

--> using a variable

Q5) what is a variable?

--> named memory location is nothing bt variable.

--> variable can be treated as a container to store data.

-->content of variable can vary and hence name variable

Q6) write a python program that adds two number and displays the sum?

--> addition.py

```
----------
a=10
b=12
sum=a+b
print("the sum is",sum)
```

Q7) how to separate a string and variable in print function ?

--> using comma(,)

Q8) how to read data from keyboard in python?

--> using input() library function

Q9) write a python program that prompts the user to enter two number add them and display result?

--> interactiveaddition.py

```
----------------------
a=input("enter first no.")
b=input("enter second no.")
sum=a+b
print("the sum of",a,"and",b,"is",sum)
```

output:-

```
--------
enter first no. 10
```

```
enter second no. 30
the sum of 10 and 34 is 1034
```

Note:- reason for above unexpected output is, eve through user enter a number form keyboard, input() function return it to the program as a string. it means "a" and "b" acting as string variable, the "+" operator not acting as arithmetic addition but is acting as string concatenation operator.

--> To address this problem, typecasting/ type conversion should be performed using int() library function.

interativeaddtion2.py

```
---------------------
a=input("enter first no.")
b=input("enter second no.")
sum=int(a)+int(b)
print("the sum of",a,"and",b,"is",sum)
```

output:-

```
--------
enter first no. 10
enter second no. 30
the sum of 10 and 34 is 44
```

OR

```
a=int(input("enter first no."))
b=int(input("enter second no."))
sum=a+b
print("the sum of",a,"and",b,"is",sum)
```

OR

```
a=eval(input("enter first no."))
b=eval(input("enter second no."))
sum=a+b
print("the sum of",a,"and",b,"is",sum)
```

Note:- eval() is a library function that converts the data into the specified type

Q10) write a python program that prompts user to enter the dimension of a room and display its area.?

--> **roomarea.py**

```
----------
length=eval(input("enter the length of a room"))
breadth=eval(input("enter the breadth of a room"))
area=length*breadth
print("area of the room is", area, "square root")
```

Q11) explain about comment in python program.?

--> textual description of the program which is ignored by python interpreter is nothing but a comment in a python program.

--> comment is a primary source of documenting the project.

--> python support single line comment only.

--> python comment starts with hash symbol (#) and ends with the physical line.

Note:-some programmer feel multiple line comments are written by enclosing between triple quotes.

Q12) what are the different type of modes of python interpreter?

1. script mode

2. interactive mode

1. writing a source code in a source file and submitting it to the python interpreter for execution is nothing but using the python interpreter in script mode.

2. submitting the python code line by line to the python interpreter through shell is nothing but using python interpreter in interactive mode.

-> no file concept in this mode.

-> once shell is closed, code lost, there for interactive mode is not used in real-time project.

-> to test the functionality of small pieces of python code we use interactive mode.

Q13) what are the fundamental data types in python?

--> python support 3 kinds of data:

1. number
2. True/False values
3. String

Number:- three categories

1. integral number(without decimal part)
2. real number(decimal/ fraction part include)
3. complex number

-> "int" data type is used to represent integers

-> "float" data type is used to represent float

-> "bool" data type is used to represent bool

-> "str" data type is used to represent strings

-> there are 5 fundamental/primitive/system defined/ pre-defined data types in python

1.int 2.float 3.complex 4.bool 5.str

Note:- python doesn’t support "char" data type. To represent even a single character also in program "str" data type used.

Q14) explain about dynamically typed feature of python?

--> compilation time, variable type is not decide in python unlike in c, c++ or java.

--> in statically types language , variable type is decided at compile time and during execution of program, only that type of data can be store.

--> in python variable types is decided based on the type of value stored in that variable during the execution of the program this is known as dynamically typed feature of python.

```
a=10 #type of a is "int"

a="hello" #type of a is "str"

a=10.5 #type of a is "float"

a=True #type of a is "bool"
```

Note:- in python, not only variable value but also variable type can be changed dynamically

Q15) how to know the variable type in python?

--> by type() library function

Q16) how to declare a variable in python ?

--> associating a variable to a data type is nothing but declaring a variable.

--> in statically typed language, we have the concept of variable declaring.

--> python is dynamically typed programming language. we don’t have the concept of variable declaration in python.

Q17) how to create a variable in python?

--> assignment statement creates a variable in python.

eg. a=10

Q18) when is variable type decided in python?

--> during the execution of program

Q19) what is an identifier?

--> name of a programming element is nothing but a identifier.

--> variable, function, class of the programming elements.

Q20) what are the rules in framing an identifier in a python program?

--> lower case alphabetes, upper case alphabetes, digits and underscore are the only allowed characters in a pytho identifier

--> identifier should not start with a digit.

--> embeded space is not allowed in an identifier

--> keyword should not be used as identifier

Note:-

--> an identifier can hace any number of character

--> even though an identifier can start with an underscore, it is not advisable(it has special meaning)

--> python is case sensitive

--> even though use can use any identifier it should reflect its purpose.

03/06/2019

Q1) which of the following variable name are valid/invalid?

total_sal	(valid)
total-sal	(invalid: hyphen not support)
salary$	(invalid: special character not support)
_n1	(valid: but not advisable)
average salary	(invalid: embedded space not allowed)
1n	(invalid: not start with digits)
While	(invalid: keywords not allowed)

Operators

Q.) what is a operator?

Ans:- an operator is an symbol that operates upon data item known as operands in order to process them.

Eg: a+40

-> in above eg, “a” and “40” are operands and “+” is operator.

Note:- an operator can be variable or can be literal in above example “a” is variable and “40” is literal.

Q.) how are operators classified?

Ans:- operator are classified based on two categories.

1. number of operands they are operating

2. type of operation they are performing on operands.

1. based on no. of operands they are operating upon, operator are three types.

a. unary operator (one data item)

b. binary operator (two data item)

c. ternary operator (three data item)

2. based on types of operation they performing on operands, operators are divided in 7 types

a. arithmetic operator

b. assignment operator

c. relational operator

d. membership operator

e. identity operator

f. logical operator

g. bitwise operator

Q.) explain about arithmetic operator?

Ans:- arithmetic operator perform arithmetic (mathematical) operation on the operands.

There are 7 arithmetic operation in python

+

-

*

/

// (floor division/ truncate division)

% (modulus operator)

** (exponent / power operator)

{+}

- “+” acts as arithmetic addition operator when both operator are number.
- “+” acts as string concatenation operator when both operands are string

  Eg. “7” + “2” #72

- “+” operator causes error when one operand is number and other one is string

{*}

- “*” acts as arithmetic multiplication operator when both operands are number.
- “*” acts as repetition operator when one operand is number and second one is string

For eg. Print(3*”hello”) #hellohellohello

{/}

- “/” is arithmetic division operator after performing division operation, it return the quotient in decimal form. Also call complete division operator

  For eg. 7/2 #3.5

{//}

- "//" perform division and return the quotient after truncating decimal part

  For eg. 7//2 #3

  -7//2 #-4

{%}

- "%" perform division and return the result in remainder.

  For eg. 7%2 #1

  6%2 #0

{}**

- "**" return the power of number.

  For eg. 3**2 #9

  5**3 #125

Q.) explain about assignment operator?

- An assignment operator assign a value to a variable.
- "=" is a simple assignment operator in python.
- An assignment statement create a variable in python program.

  A=10 #variable is created and 10 assign to it.

- Arithmetic operator along with "=" forms a compound assignment operator

  +=

  -=

  *=

  /=

  //=

%=

**=

For eg. A=10

A+=1 #11 store in a

Note:- ++ and – operator is not support in python language unlike C.

04/06/2019

Q.) explain about relational(comparison) operator in python?

Ans:- relational operator are used to compare two data items for equality, inequality, lesser and greater etc.

1. == (equality operator)
2. != (inequality operator)
3. < (less than)
4. <=
5. > (greater than)
6. >=

For eg.

```
a=10
b=20
a==b   #False
a!=b   #True
```

```
a>b    #False
a>=b   #False
a<b    #True
a<=b   #True
```

Note:- relational operator result in “Bool” value.

Q.) explain about membership operator ?

Ans:- membership operator is used to find if a data item belongs to a collection/sequence.

There are two membership operator in python.

1. “in”
2. “not in”

- “in” operator verifies whether an element is a member of collection or not
- “not in” operator verifies whether an element is not a member of collection or not.
- Membership operator value in “Bool” value.

  For eg.

```
10 in [10,20,30]  #True
10 not in [10,20,30]  #False
40 in [10,20,,30] #False
40 not in [10,20,30]  #True
```

Q.) explain about identity operator?

Ans:- identity operator used to compare the address of two variable.

There are two identity operator in python

1. is
2. is not

- identity operator return “Bool” value.
- “is” operator returns true if the address are same other wise false.
- “is not” operator return true if address are not same other wise false.

For eg.

```
a=10
b=20
a is b #false
a is not b #True
```

Note:- id() is library function used to find the address of variable.

```
C=20

id(C)  #return address
```

Q.) explain about logical operator?

Ans:- there are three logical operator in python.

1. and
2. or
3. not

- “and” and “or” used to combine two “Bool” expression(condition) result in bool value.
- “or” operator result in true when at least one operand is true.
- “and” operator result in true only when both of its operands is true.
- “not” operand used to complement Boolean value.

For eg:-

```
a=20
b=30
```

```
c=40
a<b or b>c   #True
a>b and b>c  #False
flag=True
not flag     #False
```

Flow control statement or control structure

Q.) what are the feature of structure programming language?

Or

Q.) what are the feature of structure program?

Ans:-

1. Sequence
2. Selection/branching
3. Iteration/looping

Q.) explain about “sequence” feature of python program?

Ans:- if the program is executed sequentially line by line without jumping to other part of program, it known as “sequence” feature.

By default, python program has sequential feature. Programmer need not to anything to get this feature.

05/06/2019

Q.) explain about “selection” feature of python program?

Ans:- the ability to select a portion of a code & execute only when some condition has been satisfied it is also known as "branching" feature.

- **Selection feature is not available to python "implicitly"**
- **"if statement" is used to implement selection/branching in python program.**

Q.) what are the different control flow statement in python?

Ans:-a statement in python that control the order of execution of program is nothing but a control statement.

- **flow control statement also known as control structure.**
- **There are three control structure in python.**

1. **if statement**
2. **while statement**
3. **for statement**

Q.) what are the different forms of if statement?

1. **Simple if**
2. **if.....else**
3. **if.....elif.....else**
4. **nested if**

Q.) explain about "simple if" statement?

Ans:- whenever some condition is satisfied, if we want to select a portion of code for execution, "simple if" statement is used. When condition is not satisfied nothing is performed by "simple if".

Syntax of simple if statement

```
if <text_expression>:<statement>
```

or

```
if <condition>:
 <statement>
```

Note:- where only one python statement is required to be select using if statement, we use first syntax of if statement.

- ❖ When multiple statement are to selected, when the body of if has more than one statement, we should not use first syntax, we have to use second syntax.
- ❖ Body of the of execute when evaluate value is true.
- ❖ When value is false if not executed.
- ❖ Flow chart

Q.) write a python program that read a number from keyboard and display whether is an even number?

even.py

```
n=int(input("enter a number"))
if n%2==0:print(n,"is even number")
```

Q.) how to create a body of if statement?

Ans:- using indentation (leaving the space left side)

Q.) write a python program that read a number from keyboard and display whether is an even no. or odd no.?

evenodd.py

```
----------

n=int(input("enter a number"))

if n%2==0:

 print(n,"is even no.")

if n%2==1:

 print(n,"is odd no.")
```

06/06/2019

Q.) what is the limitation of above program?(evenodd.py)

Ans:-in the above program there are two mutually exclusive condition to verifies. Nature of simple if is after verify the condition it will not stop verifying other condition are to be verified, we should not use simple if statement.

Q.) what is the output of following:

```
    a=30

    b=20

    c=10

    if a>b or c>b:
```

```
    a=1
    b=2
   c+=3   #compound assignment
   print(a,b,c)   #1 2 13  #use dry run
```

Q.) what is the output of following program?

```
   a=10
   b=20
   c=30
   if a>b or b>c:
    a=1
    b=2
   c+=3   #compound assignment
   print(a,b,c)   #10 20 33
```

Q.) how to overcome the limitation of “evenodd.py”?

Ans:- using if.....else statement.

Q.) explain about if.....else statement?

Ans:- whenever two mutually exclusive condition are to be verified for branching python code we go for “if.....else statement”.

Syntax of if.....else statement

```
--------------------------------
if <condition>:
 <statement(s)>
```

else:

<statement(s)>

- **when the condition is evaluate to true body of the "if statement" is executed body of "else" is not all executed.**
- **when condition evaluated to false body of the if statement is not executed. Instead body of "else" statement is executed.**
- **Flow chart**

Q.) modified the previous "evenodd.py" so as to use if.....else statement?

Evenodd2.py

```
n=int(input("Enter a number"))
if n%2==0:
 print(n,"is even")
else:
 print(n,"is odd")
```

Q.) explain about if...elif...else statement?

Ans:- whenever more than 2 mutually exclusive condition are to be verified for selection(branching). We go for if...elif...else statement.

Syntax

```
if <condition>:
 <statement(s)>
elif <condition>:
 <statement(s)>
elif <condition>:
 <statement(s)>
-------
-----#any number if elif statement
else:  #optional
 <statement(s)>
```

Note:- in this form of if statement, we have only one if statement any number of elif statement(according to project requirement) and only one else that is optional.

- ❖ When test expression of if is evaluated to True, body of if is executed. In such case no other elif or else is executed.
- ❖ When if condition is evaluated to false, chance for elif, elif evaluated expression is true then elif execute and next elif or else not execute.
- ❖ elif condition is false then else body execute after that program code execute.

Q.) write a python program that read a number for keyboard and display if it is +ve or –ve?

Positivenegativ.py

```
-----------------
```

```
n=int(input("Enter a number"))
if n>0:
 print(n,"is +ve")
elif n<0:
 print(n,"is -ve")
else:
 print(n,"is neither –ve not +ve")
```

Q.) write a python program that read a digits from the keyboard and display in words?

Digits.py

```
n=int(input("Enter a number"))
if n==0:
 print("ZERO")
elif n==1:
 print("ONE")
elif n==2:
 print("TWO")
elif n==3:
 print("THREE")
elif n==4:
 print("FOUR")
elif n==5:
 print("FIVE")
```

```
elif n==6:
 print("SIX")
elif n==7:
 print("SEVEN")
elif n==8:
 print("EIGHT")
elif n==9:
 print("NINE")
else:
 print("enter correct digit")
```

07/06/2019

Q.) explain about nested if statement?

Ans:- an if statement placed in the body of another if statement is nothing but a nested if statement.

Whenever condition is required to be checked at different level, we go for nested if.

Syntax:-

```
if <condition>:
 if <condition>:
  <statement(S)>
```

Q.) write a python program that read number from keyboard and display if it is +ve even or +ve odd and –ve even or -ve odd.

Positivenegativeevenodd.py

```
n=eval(input("Enter a number"))
if n>0:
 if n%2==0:
  print(n,"is +ve even number")
 else:
  print(n,"is +ve odd number")
elif n<0:
 if n%2==0:
  print(n,"is -ve even number")
 else:
  print(n,"is -ve odd number")
else:
 print(n,"is neither negative nor positive")
```

Q.) explain about iteration feature in python?

Ans:- whenever a task is required to be preformed repeatedly at the same place of the program, iteration feature is used. It also known as looping.

In python there are two control statement are give to implement iteration.

1. **For statement**
2. **While statement**

Q.) explain about "for" statement.

Ans:-used to implement iteration/ looping in python.

“For” is used to loop over a collection. ie to access the element of collection “for” loop is used in python.

Syntax of for loop

```
for <variable> in collection/sequence :
    <statement(s)>
```

Q.) write a python program to access the element of a list/collection.

Ans:-

```
#listtravelsal.py
list=[10,20,30]
for item in list:
 print(item)
```

output:

```
10
20
30
```

Q.) how many times body of for loop is executed?

Ans:- as many times as there are items in collection.

Q.) write a program to display sum of all number.

Ans:-

```
#sumlist.py
list=[10,20,30]
sum=0
for i in list:
```

```
 sum +=i
print(sum,“is list addition”)
```

Q.) explain about “while” statement in python?

Ans:- while statement is a control statement in python in which is use to implement iteration/looping in python.

Syntax

```
while condition:
 <statement(s)>
```

As long as the condition is true so long the body of while loop is executed.

Once the condition is executed to false, while loop is execute.

Q.) what is the output of following program.

```
i=1
while i<5:
 print(i)
```

output

1

1

1

1...so as long

- **1 is printed infinite no. of times.**

- **Loop counter i is not update in the body of while loop. As a result the condition of the while statement becomes true for execute that is above while loop has become an infinite loop.**

09/06/19

Q.) explain about bitwise operator?

Ans:- bitwise operator acts on bits.

There are 6 bitwise operator in python.

1. **AND (&)**
2. **OR (|)**
3. **XOR (^)**
4. **NOT (~)**
5. **Left Shift (<<)**
6. **Right Shift (>>)**

For eg.

```
a=20

b=4

a&b #4

a|b #20

a^b #16

~a  #-21

a<<2   #80

a>>2   #5
```

- ❖ AND (&) operator acts on two bits result in 1 only when both the bits is 1, otherwise it return 0.
- ❖ OR (|) operator acts in two bits and result in 1 when at least one bit is 1, otherwise it return 0.
- ❖ XOR (^) operator acts on two bits and result in 0 when both bits are same, it result 1 when both are different.
- ❖ NOT (~) operator flips(complement) the bit. It 1 is there result in 0 and it is 0 result in 1.
- ❖ >> (right shift) operator shift the specified no. bits to the right.
- ❖ << (left shift) operator shift the specified no. of bits to the left.

Q.) explain about ternary operator?

Ans:-ternary operator is an if else operator.

Syntax

<expression1> if <condition> else <expression2>

- ❖ When condition is true expression1 is call when condition is false expression.
- ❖ Ternary operator main purpose is to replace the if else statement and reduce the code.

Ternaryexample.py

```
n1=int(input("Enter a number"))

n2=int(input("Enter a number"))

min= n1 if n1<n2 else n2
```

print(min,“is smaller”)

ternaryexample2.py

feedback=80

print(“good” if feedback>80 else “average”)

Q.) explain about operator precedence?

Ans:- priority of order of execution of an operator in preference to other operator in an expression is nothing but operator precedence.

Following list specified the operator in descending order of their.

()

******

***,/,%,//**

+,-

==,!=,<,<=,>,>=, in, not in, is, is not

Logical not

Logical and

Logical or

Associativity guide the order of execution of the operator equal precedence in an expression mostly it is left to right except in case of exponent operator(**).

Q.) explain about type casting.

Ans:-converting the data item of one fundamental type into the other type is type casting/ type conversion/ type coercion.

5 built in function are provided in python for type casting of primitive type(that represent single data item)

int(), float(),complex(), bool(), str()

int() :- is built inn function used to convert other type into int type.

Eg

int(10.7) #10

int(Flase) #0

int(True) #1

int(“10”) #10

int(“10.5”) #error

int(“ten”) #error

Note:- complex data type cannot convert into int data type.

float() :- is built inn function used to convert other type into float type.

Eg.

float(10) #10.0

float(“10.4”) #10.4

float(True) #1.0

float(Flase) #0.0

float(“abc”) #error

float(30+4j) #error

Note:- complex data type cannot convert into int data type.

bool() :- is built inn function used to convert other type into bool type.

Eg.

Bool(“”) #False

Bool(“ ”) #True

Bool(0.0002) #True

Bool(“hello”) #True

str() :- is built inn function used to convert other type into str type.

For eg.

str(10) #’10’

str(10.5) # ’10.5’

str(True) #’True’

str(2+3j) #’2+3j’

Q.) explain about immutability of data type.

Ans:-Immutability(non modifiability) is a feature in which the content of a memory location can't be modified once it is created and a data item is stored.

=>int, float,complex,bool & str types are immutable in Python

I.e. for these type of variables once assignment statement is written, memory is allocated and data is stored, even if we assign different value later, a new memory location is created but earlier location's content is not modified.

a 20

id(a) # prints the address pointed to by a

bra # two references pointing to the same memory location

id(b) # prints the address pointed to by b

a is b# prints True

a=30

id(a) #prints the address pointed to by a

a is b #prints False

Note: In Python, variables are not containers of data. They are lables to the containers(objects)that store data.

String Handling

Q)What is a string?

=> An immutable, indexed sequence of unicode characters that is represented as a single entity is nothing but a String in Python.

>A string is represented using "str" data type in Python.

=>Textual representation of a data item in a Python program is a string.

Note:- Unicode characters fecilitate internationalization(i18n)

For eg.

s=”Python”

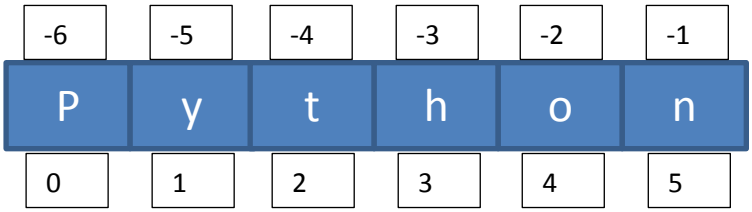
OR

S=’Python’

OR

s=””Python””

s=”””Python”””



Q.) how to access the character of string?

Ans:-

1. Using index
2. Traversal
3. Slicing

Using index

S=”python”

S[2] #’t’

S[0] #’p’

S[-1] #’n’

using traversal

accessing each element of a string one by one sequentially using a loop called string traversal.

```
s=”python”
for i in s
 print(i)
```

output:

p

y

t

h

o

n

or

```
s=”python”
for i in s
 print(I,end=””)  #by default end=”\n”
```

output:

python

or #using while loop

```
s=”python”
size=len(s)
```

```
c=0
while c<size:
 print(s[c])
 c+=1
```

using slicing

extracting a portion to string is known as slicing

slicing operation will be following formate

[start:end:step]

Start specified from where slicing will start.

End specified slicing should be up to what index(excluding last).

Step specified the increment the index. Default is 1.

```
s=’hello’
s[::2]     #’hlo’
s[-2::]    #’lo’
s[::-1]    #’olleh’
```

Q.) what is the output of the following piece of code?

```
s=’hello’
s[1]=a
print(s)
```

error,string immutable

Q.) write a python script that takes a string from the user and display if it is a palindrom?

```
#palindrome.py

s=input(“enter a no.”)

s1=s[::-1]

if s==s1:

 print(s,”is palindrome”)

else:

 print(s,”is not palindrome”)
```

Q.) how to delete a string

Ans:- by using del keyword.

Character of string cant be deleted as string is immutable.

A string variable can be deleted using del keyword.

For i.e.

```
s=’hello’

del s  #memory deallcated
```

Q.) how to know the number of character in string?

Ans:- using library function len()

Q.) how to find if a sub string is present in a string or not?

Ans:- using membership operator.

For i.e.

s=’hello’

“al” in s #False

“hell” in s   #True

10/06/19

Control structure

Q.) what is the output of following code?

```
i=1    #loop counter creation
while i<5:
 print(i)
 i+=2  #loop counter update
```

output:

1

3

Q.) write a python program to print a multiplication table of user choice up to 10 multiples?

Ans:- mtable.py

```
t=int(input(“which multiplication table to print”))
i=1
while i<=10:
 product=t*i
 print(t,”X”,i,”=”,product)
 i+=1
```

Q.) what is a nested while statement?

Ans:- a while statement placed within the body of another while statement is nothing but nested while loop.

i.e.

```
while <condition>:
  while<condition>:
    <statement>
```

For every iteration of outer while inner while complete its all iteration.

Q.) write the output of following code:

```
i=1
j=2
while i<4:
 while j<6:
  print(i,j)
  j+=2
 i+=1
```

output:

1 2

1 4

Q.) what is the output of following python script.

```
i=1
while i<4:
 j=1
```

```
 while j<5;
  print(i,j)
  j+=1
 i+=1
```

output:

```
1 1
1 2
1 3
1 4
2 1
2 2
2 3
2 4
3 1
3 2
3 3
3 4
```

Q.) write a python program that print multiple table of user specified range?

Ans:- mtables.py

```
t1=int(input(“from which table”)
t2=int(input(“up to table”))
while t1<=t2
 i=1
 while i<=10:
```

```
  product=t1*i
  print(t1,”X”,i,”=”,product)
  i+=1
 t1+=1
```

Q.) explain about “break” statement ?

Ans:- break statement used to exit loop forcefully.

Q.) what output of following code:

```
i=1
while i<=10
 print(i)
 if i==4
  break
 i=i+1
```

output:

1

2

3

4

Q.) explain about continue statement?

Ans:- “continue” statement skip the current iteration of loop. Remaining iteration will continue.

Q.) what output of following code:

```
i=1
```

```
while i<=4
 i+=1
 if i==3
  continue
 print(i)
```

output:

2

4

5

Q.) how to create a infinite loop with while loop?

Ans:-

```
while True:
    <statement>
    if condition:
        break
```

11/06/2019

Q.) what is the output of following program:

Continueexample.py

```
items=[100,249,1000,500]
for item in items:
 if item>600:
    print("item is greater than 600 can't be
    processed")
```

```
    continue
 print(item,“processed”)
```

ans:- 100 processed

      249 processed

      item is greater than 600 can’t be processed

      500 processed

Q.) explain about “else” statement associated with loop.

Ans:- in python, “else” statement is not only associated with “if” statement but also associated with “while” and “for” statement.

“else” statement when associated with “for” statement execute when loop is exited naturally without break statement being execute.

Q.) what is the output of following program:

```
#elseexample.py
items=[100,200,400,500]
for item in items:
 if item>600:
    print(“item is greater than 600 can’t be processed”)
    break
 print(item,“processed”)
else: print(“all item are processed”)
```

Ans:- 100 processed

      200 processed

400 processed

500 processed

Q.) what is the output of followings:

```
#elseexample2.py
items=[100,2490,4000,500]
for item in items:
 if item>600:
    print("once item is greater than 600 than processing is stoped")
    break
 print(item,"processed")
else:print("all item are processed")
```

Ans:-

100 processed

once item is greater than 600 than processing is stoped.

Q.) explain about "pass" statement associated with loop in python?

Ans:-"pass" statement can be written in the body of for loop or while loop.

"pass" statement doesn't do anything.

"pass" statement acts as a place holder indicates that as of now we are not interested in implementing the body & later we may.

"pass" statement is null statement used to create null body for loop statement/function/classes.

#passexample.py

```
list=[10,20,30]
for item in list:
 pass
```

Function

Q.) what is a function?

Ans:- named block of code that performs a specific task when called, is nothing but a function.

Q.) why function concept?

Ans:- function fecilated

1. Code reusability
2. Modularity
3. Better code maintenance

Q.) how are function classified?

Ans:-1. Library function 2. User define function

Q.) what is library function?

Ans:- a pre-created function comes along python installation is nothing but a library function. Also known as standard function.

Which task are commonly required in all type of application those task are performed in pre-defined function known as library functions.

12/06/19

Q.) what is user defined function?

Ans:- our own created that is programmer created function is nothing but a user defined function.

- A python application is nothing but a collection of user define function.

Q.) what is the syntax to create(defined) a function in python?

Ans:- def <identifier>(parameter):

<statement(s)>

- A function has two parts.
  - Function header
  - Function body
- Function header has 4 parts.
  - “def” keyword
  - Name of function
  - Parameter list(optional)
  - Colon
- Task performing code of the function a group of statement ,it is known as the body of function.
- Function body created using indentation.
- Function body have optional return statement.

Q.) write a user defined function that performs addition of two number taking number from keyboard and display the sum?

Ans:-

```
def add():
 n1=int(input(“Enter first number”))
 n2=int(input(“Enter second number”))
 sum=n1+n2
 print(“the sum is”,sum)
```

Q.) what is the output of following python program?

```
def add():
 n1=230
 n2=20
 sum=n1+n2
 print(“the sum is”,sum)
```

Ans:- no output

Reason:- in above program function is defined not called.

Q.) what is the output of following python program?

```
def add():
 n1=20
 n2=30
 sum=n1+n2
 print(“the sum is:”,sum)
add()  #function call
```

Ans:- the sum is:50

Q.) what are the “parameters” in function?

Ans:-

- Variables specified within the function header during function definition are nothing but parameter.
- Caller use parameter to supply the input data to the function. i.e. parameter means with which data are input a function for processing.

Q.) write a program that perform addition using function and that takes parameter .

```
def add(a,b):  # a & b parameter
 total=a+b
 print("the sum is:",total)
add(10,54)    # 10 & 54 arguments
```

13/06/19

Q.) what are arguments?

Ans:- values supplied to a function during function call are known as arguments.

Note:- parameter and arguments are interchange by used by programmer.

Q.) how are function categorise based on parameters?

Ans:-

1. Non-parameterised function
2. Parameterised function

- Function they don't take parameter known as non-parameterised function.
- Function are have parameter known as parameterised function.

Q.) modify the addition5.py so as the python program to read the number from keyboard and supply them to add function.

Ans:- #addition6.py

```
def add(a,b):  #formal parameter
 total=a+b
 print("sum is :",total)
```

```
n1=int(input(“Enter the first no.”))

n2=int(input(“Enter the second no.”))

add(n1,n2) #actual parameter
```

Note:- during function call, if supplied values are literal they are simply called as argument. If supplied values are variable then arguments also known as “actual parameter”. In such case, function definition time specified parameter is formal parameter.

Q.) modify the “addition5.py” program so as “add” function to return process data to caller?

Ans:-#addition7.py

```
def add(a,b):

 total=a+b

 return total

c=add(10,20)

print(“sum is :”,c)
```

14/06/19

Q.) write a python program with two user define function:

1. that calculates the area

2. that calculates the cost of flooring

Ans:-#flooringcost.py

```
def roomarea(length,breadth):

 area=length*breadth

 return area
```

```
def flooringcost(length,breadth):
 area=roomarea(length,breadth)
 cost=100*area
 print("flooring cost RS.",cost)
length=int(input("Enter length: "))
breadth=int(input("Enter breadth: "))
flooringcost(length,breadth)
```

Q.) when to use return statement in the function?

Ans:- after data processing, to handover data to the caller return statement is used in a function. Function caller get that processed data and use it for further processing.

Q.) what is the output of following:

```
def add(a,b):
 sum=a+b
 print("the sum is: ",sum)
print(add(10,20))
```

output:

```
the sum is: 30
None
```

Note:- when there is no explicit return statement in function body, every python function by default return "None" to the caller.

Q.) how are variable classified based on the place of creation?

1. local variable

2. global variable

Q.) what is local variable?

- A variable created inside a function body is known as local variable.
- Local variable accessible only within the function where they are created. Outside the function not access and not access in other function or python script.
- Scope of local variable is in that function is created.
- Life time of a local variable for the duration of the execution of that function in which that local variable is created.

Note:- life of local variable is nothing but allocation & deallocation.

```
def deposit():
 bal=500   #local variable
 print(“deposit balance is: ”,bal)
def displaybal():
 print(“balance is: ”,bal)   #error
deposit()
displaybal()
print(“balance is: ”,bal)    #error
```

Q.) what is global variable?

- created outside the function
- access all function in a program(read-only)
- scope of variable entire application.
- Life time is that application.

```
bal=500
```

```
def deposit():
print(“deposit balance is: ”,bal)
def displaybal():
 print(“balance is: ”,bal)
deposit()
displaybal()
print(“balance is: ”,bal)
```

17/06/2019

Q.) what is the output of following:

```
balance=500
def deposit():
 balance=1000
def displaybalance();
 print(“balance is Rs.”,balance)
deposit()
displaybalance()
```

output:-

balance is Rs. 500

- As global variable are accessible in all function of the application, output should have been, “balance is Rs. 1000” in deposit function “balance” variable value is made 1000.
- Whenever a global variable value is modified in user define function, normally local variable is created but global not modified.

- In a user define function, if we want to modify global variable value, we need to use keyword global.

Q.) what is the output of following:

```
balance=500
def deposit():
 global balance
 bal=1000
def displaybalance():
 print(“balance is Rs.”,balance)
deposit()
displaybalance()
```

output:-

balance is Rs. 1000

Q.) how are function arguments classified in python.

1. Positional arguments
2. Name parameters/ keyword parameters
3. Default argument
4. Variable length positional arguments
5. Variable length keyword arguments

Q.) explain about positional arguments?

--> positional arguments are those arguments supplied to a function during function call based on the number and order of parameters specified in the function definition.

Note:- if arguments order is change in case of positional arguments, unexpected result given by the function.

```
#positionalarg.py

def displayAccountInfo(accno,name,balance):

 print(“ACCNO”,accno)

 print(“NAME”,name)

 print(“BALANCE”,balance)

diplayAccountInfo(100001,“ram”,1000)
```

Q.) explain about keyword arguments.

- Keyword arguments are those arguments supplied to a function during function call as parameter name, value pair.
- In case of keyword arguments, number of arguments matter but not order.
- Whenever more number parameters are their for a function, keyword arguments are very useful to avoid unexpected result from the function

```
#keywordarg.py

def display(accno,name,balance)

 print(“ACCNO”,accno)

 print(“name”,name)

 print(“balance”,balance)

display(name=“ram”,balance=1000,accno=100001)
```

Note:-when both positional arguments are keyword argument are used during call function, keyword should come after the positional.

Q.) explain about default arguments.

- Default arguments are those arguments from which function definition initital value are assigned to the parameters.
- For eg

```
def display(accno,name,balance=1000) #definition
```

- When default function arguments are used, we need not to supply value for that parameter during function call.
- For eg

```
display(10001,“ram”)    #balance taken 1000
```

- We can supply value to default arguments also when need during function call supplied value are overwrite the default value.

```
#default.py

def display(accno,name,balance=1000)

 print(“ACCNO”,accno)

 print(“name”,name)

 print(“balance”,balance)

display(100001,“rama”) #balance dislay 1000
```

Note:- after default argument, non-default argument should not be specified in function definition.

Q.) explain about “variable length positional arguments”.

- Variable length positional arguments are specified during function definition as follows:

*Parameter

- For eg

```
def add(*n)
```

- During function call, any number of (0-n) arguments can be supplied to the function and hence the name.
- Internally arguments are stored in a built in data structure, “tuple”

```
#vararg.py
def add(*n)
 sum=0
 for i in n:
 sum=sum+i
 print(sum)
add()
add(10,20)
add(10,20,30)
```

Note:-before vararg we can supply positional arguments also but after vararg if we want to supply normal argument it should be only keyword.

18/06/2019

Q.) explain about variable length keyword argument?

- variable length keyword argument are those argument which are supplied as key, value pairs to the function during function call.
- Variable length keyword argument parameter is supplied using double asterisk during function definition.

- For eg.

```
def n1(**n)
```

- Variable length keyword argument are stored in dictionary.
- #kwargs

```
def displayAccInfo(**accountInfo):
 for key,value in accountInfo.items():
  print(key,“:”,value)
displayAccInfo(accno=100001,name=“rahim”)
displayAccInfo(accno=100201,name=“ram”,balance=80000,emailid=”abc@gmail.com”,mob=9809809808)
```

Output:-

accno:100001

name:rahim

accno:100201

name:ram

balance:80000

emailid;abc@gmail.com

mob:9809809808

Q.) what is function aliasing?

Ans:- giving another name (second name) to a function is called function aliasing.

For eg:

```
def f1()
 print(“in f1()”)
```

```
f2=f1  #function aliasing
f2()   #call
```

Q.) how to return multiple value from the function?

Ans:- by specifying the comma separated left of variable in the return statement, multiple values can be returned from a function in python.

For eg:- return a,b

Q.) what is the output of following:

```
def swap(a,b)
 b,a=a,b
 return a,b
x=100
y=200
print("before swapping x=",x," y=",y)
x,y=swap(x,y)
print("after swapping x=",x," y=",y)
```

Q.) does python support call by value to call by reference?

- Everything is an object in python.
- When a function call by supplying an arguments, it is always object reference, there for during a function call in python, it is always technically call by object reference.
- However, during python function call, if an immutable object's reference in supplied as argument it is call by value as changes mode through formal parameter doesn't get reflected in the object reference by actual parameter.

- If a mutable object reference is supplied a argument while calling a function. It is call by reference as changes made through formal parameter gets reflected in the object reference by actual parameter.
- For eg:- if a built in data structure list is supplied as argument during function call it is “call by reference” as list is mutable.
- If primitive variable is supplied as argument (int,float,double,bool,complex) it is “call by value” as they are immutable.
- #callbyreference.py

```
def f1(a):
 a=20
b=10
f1(b)
print(b) # 10
```

- #callbyvalue.py

```
def f1(a):
 a[0]=200
 a[1]=300
b=[10,20]
f1(b)
print(b) # 200 300
```

Q.) what is nested function?

Ans:- if a function is defined within the body of another function. It is called as nested function.

Eg:

```
        def f1(): #enclosing/outer
          def f2():#inner/nested
             print("inner")
```

when a function has larger body to modularise that function body, nested function are used.

19/06/19

Q.) what is the output of following:

```
#nested1.py
def f1():  #enclosing function
 x=20
 def f2(): #nested function
  x=40
  print("x=",x)
 f2()      #nested function call
 print("x=",x)
f1()       #outer function call
```

Ans:-

```
       x=40
       x=20
```

Note:- nested function can access the local variable of outer function for read only.

If nested function tries to modify the outer function variable value, instead of making modifications, a new variable is created within the nested function.

Q.) how to modify outer function variable value in nested function?

Ans:- make use of the keyword “nonlocal” to prevent new variable creation and then modify.

Q.) what is the output of following:

#nested2.py

#nested1.py

```
def f1():  #enclosing function
 x=20
 def f2(): #nested function
  nonlocal x
  x=40
  print(“x=”,x)
 f2()      #nested function call
 print(“x=”,x)
f1()       #outer function call
```

Ans:-

```
       x=40
       x=40
```

Q.) what is recursive function?

- A function that is calling itself is a recursive function.
- Certain computing problem are elegantly slove by recursion.

#factorial.py

```
def findfact(n):
 if n==0 or n==1:
  fact=1
 else:
  fact=fact*findfact(n-1)
 return fact
n=int(input("Enter a number"))
print("factorial of ",n,"is: ",findfact(n))
```

Q.) what is an anonymous function?

- Name less function are known as anonymous function.
- Anonymous function also known as lambda function.
- Body of a anonymous function is not associated with any identifier.
- Lambda function are meant for short form one time usage.

Note:- in python, a function can be supplied as arguments to another function similarly a function, a function can return another function in return statement.

- Most of times, lambda function are used to supply as arguments to other functions and as return type of other function.
- Syntax to define an anonymous function

```
labda arguments:expression
```

for eg

```
labda x,y:x+y
```

- In lambda function we can have only one expression we can have any number of arguments. “expression” is the body of the anonymous function we cant make function call in its body.

```
#lambdaexample.py

product=lambda x,y:x*y    #product in the reference of the function object

print(“Product =”,product(4,5))  #using the reference invoking the lambda function.
```

20/06/2019

Modules

Q.) what is module?

- Python source file (.py file) is nothing but a module.
- A module is nothing but a collection of variable and user define function that is made available to other parts of the same function.

```
Module= global variable + user defined function +
        script
```

Q.) why module concept?

- Module facilitates code reusability and modularity.

Q.) how are modules classified?

1. Application modules (.py file of same project)
2. Standard / built-in modules (that came with python installation)
3. Third party modules (other than first two)

Q.) how to create a module in python?

- Python source file creation is nothing but module creation. i.e. when we create a “.py” file a module is created.

Q.) how to make other modules available in a python program?

- Using import statement.

Q.) how to make use of python library function in a python program?

- Step1:-make that library function available to the program by importing the module to which that function belongs.
- Step2:-call that library function.

Q.) write a python program to find a square root of number?

```
#squrtexample.py

    import math

    n=eval(input(“enter a number”))

    sr=math.sqrt(n)

    print(“the square root of ”,n,“is”,sr)
```

Q.) how are print(),input(),type(),eval() etc. library function available to a python application?

- All these library function belong to a standard module “__builtins__”
- This is implicitly available to all the python applications.
- print(),input(),type(),eval() function belong to this module.

- Therefore, without the need of importing any module these library function can be called directly.

21/06/19

Q.) what happens in the background when “import” statement is encountered by python interpreter?

```
    #E:\today\m1.py
def f1():
 print(“in m1.f1()”)
def f2():
 print(“in m1.f2()”)
print(“this is the script”)
```

```
    #E:\today\m2.py
import m1
m1.f1()
```

- When “m2.py” executed
  #E:\today\m2.py
- Python interpreter executes the import statement in “m2.py”. the following thing happen.

1. Imported module’s source file, i.e. “m1.py” is compiled to “.pyc” file this file is created in folder by name “__pycache__”. This folder is created in today folder.
2. All the user defined function and global variable of the imported module are made available to the python program. i.e. current module “m2.py”.

3. Script of imported modules source file. i.e. m1.py is executed.

Note:- “.pyc” file contains byte code, byte code is nothing but intermediate instruction understandable to any “pvm”.

Q.) how to prevent the execution of the script of the imported module during execution of python program?

- For every “.py” file, i.e. python module, python interpreter maintains an attribute, “__name__”
- Whenever that module is directly executed, python interpreter stores a string “__main__” in that attribute of that module.
- Whenever a “.py” file is executed indirectly(because of other file’s imported statement), “__name__” attribute get module name as the value.
- By checking this attribute value scripting of imported module can be prevented from execution.
- Inn fact in a real-time project in every “.py” file script is written under “if” condition only.

```
if __name__== “__main__”

    <statement(s)>
```

22/06/19

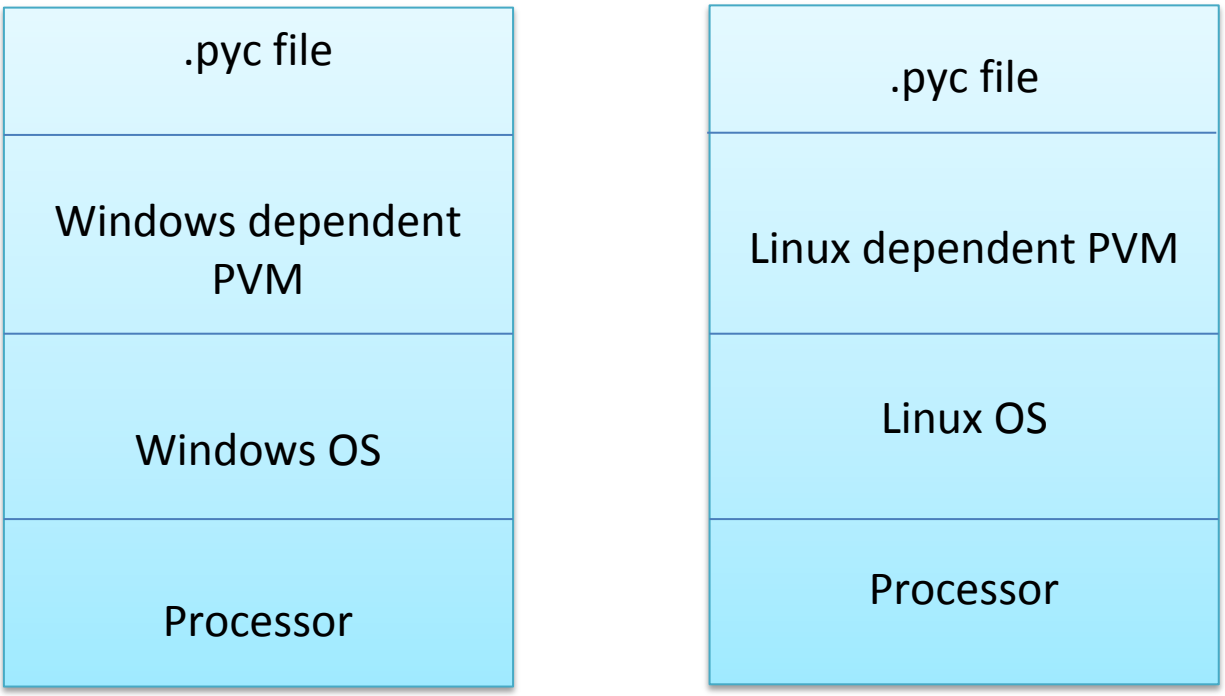
Q.) how are python program are OS independent?

- Three thing to the OS independent of python program.
  - Compiler
  - “.pyc” file
  - OS dependent PVM

- Cpython compiler compiles “.py” file to same “.pyc” file in any os environment.
- “.pyc” file contains byte code which is nothing but intermidiatory instruction understandable to any PVM.

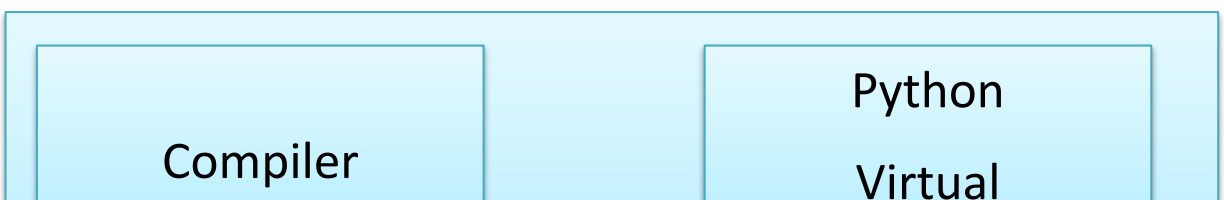
Note:- Python Virtual Machine are different for different OS.

- PVM executes the python program. PVM is part of interpreter.
- PVM translate (interprets) byte code into underlying OS understandable machine code to be given to processor.



Windows machine Linux machine

Q.) who executes python program. Python virtual machine? or python interpreter?



PVM executes the python program. PVM is part of interpreter.

Q.) how to explicitly compile a python program?

- By using library function of “compileall” built-in module.

1. compile_file() function used to compile one python file.
2. compile_dir() function used to compile all the python file of directory.

Q.) what are the different ways of importing modules into the python file.

- For eg. in “m1” module (.py file) assume f1() and f2() are two functions.
- All possible ways of importing modules.

1. import m1

- with the above type of import statement “m1” name only is placed in current module’s name space(current program’s private symbol table). Therefore, dot notation is required to access its members.

```
m1.f1()
m1.f2()
```

2. `import m1,m2,m3  # 3 module at a time import`
3. `import m1 as m  # module aliasing`

- now onward , member of m1 are accessible only through alias name.

```
m.f1()
m.f2()
```

4. `import m1 as m, m2 as mm, m3 as mmm`
5. `from m1 import f1`

- here, f1 is places directly in the current module namespace and therefore, can be accessed directly without dot notation.

6. `from m1 import f1,f2`
7. `from m1 import *`

- all member of m1 are placed in current module's namespace . without dot notation they can be accessed. Danger is if anem conflicts arise, member name are over written in symbol table and are not available in the program.

Note:- this way of importing is not advisable.

8. `from m1 import f1 as function1.`
9. `from m1 import f1 as function1, f2 as function.`

Q.) explain about dir() function:

- "dir()" is a library function of "__builtins__" modules.

- There are two versions of this function.

1. Zero argument version. This function returns the name of all the attributes(function name and variable name) of the current module python interpreter maintained attribute name also it return.
2. Parameterised version this version takes “module name” as argument and return that module’s attributes (both user defined and built-in).

Q.) explain about reloading of a module.

- A module is loaded once with an import statement into memory irrespective of the number of import statement.
- Once a module is loaded into memory, after its loading if any changes are made to that module those changes are not reflected in the importing program. In such case, that module has to be reloaded using reload() library function of “importlib” standard module.

**********END**********